Redes convolucionales

Métodos Generativos, curso 2026-2027

Guillermo Iglesias, guillermo.iglesias@upm.es
Edgar Talavera Muñoz, e.talavera@upm.es

7 de noviembre de 2026

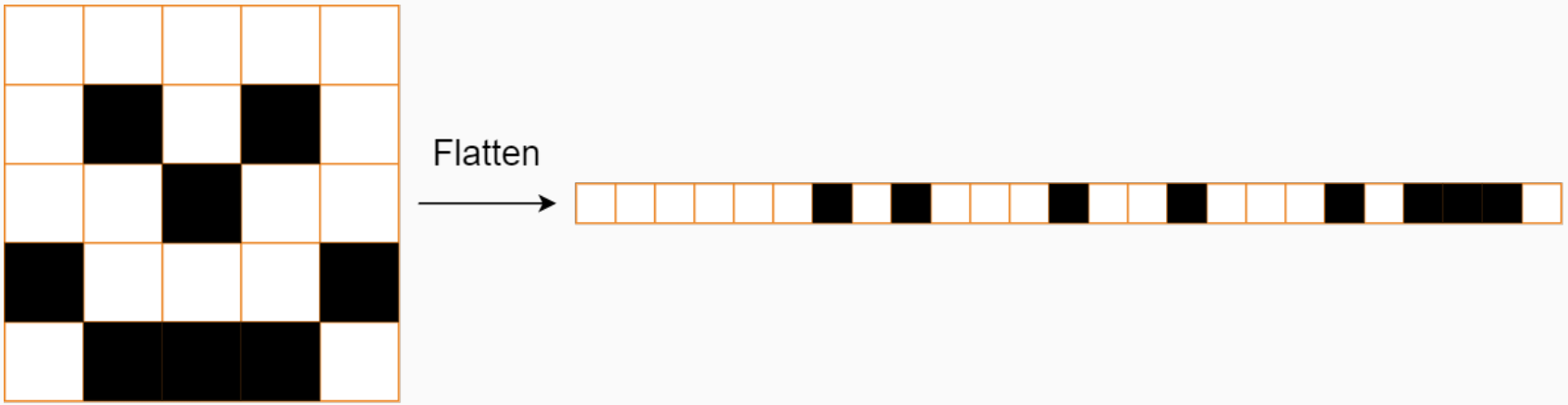
Escuela Técnica Superior de Ingeniería de Sistemas Informáticos | UPM



Motivación

Problemas del perceptrón

Como anteriormente se ha visto, una arquitectura de perceptrón es capaz de tratar con imágenes. Para ello las matrices bidimensionales o tridimensionales son transformadas a un vector unidimensional con la operación de "flatten".



Problemas del perceptrón

El principal inconveniente de esta aproximación es que se pierde toda la información espacial de la imagen.

Esto hace que se pierdan las relaciones de distancia y color.

Otro problema es la enorme magnitud de las redes creadas de esta manera.

512x512x3 píxeles = 786.432 neuronas entrada

Redes convolucionales

Las redes neuronales convolucionales surgen para adaptar las redes neuronales al tratamiento de imágenes.

Los principales beneficios de su uso son los siguientes:

- Aprovechamiento de la información espacial.
- Reducción del número de parámetros.
- Invarianza aprendida de los datos (traslaciones, escalas, deformaciones).

Fundamentos de las redes convolucionales

Operación de convolución

La operación de convolución consiste en la combinación lineal de una ventana de píxeles de una imagen.

Para ello hay dos elementos fundamentales:

- Imagen de entrada: Una matriz bidimensional de datos (normalmente normalizada a [-1, 1] o [0, 1]).
- Filtro o kernel: Una matriz (normalmente de 3x3 o 5x5) con la que se realizará la combinación lineal de los elementos de la imagen.

Input

2	1	4	0
1	2	2	0
3	1	2	1
0	0	-1	1

Kernel

0	1	-1
0	1	2
0	1	0

Operación de convolución

La salida se calcula haciendo una combinación lineal de cada región de la imagen. De esta manera la salida contiene la activación de cada zona de la imagen.

Esta región que el kernel es capaz de observar se conoce como campo receptivo.

- Capa 1: kernel 3×3, campo receptivo 3x3; Capa 2: kernel 3×3 aplicado sobre la salida de la primera capa, campo receptivo de cada neurona es 5×5; ...

Input

2	1	4	0
1	2	2	0
3	1	2	1
0	0	-1	1

Kernel

0	1	-1
0	1	2
0	1	0

Output

4	8
5	5

Ejemplo de convolución 2-D

f

2	1	4	0
1	2	2	0
3	1	2	1
0	0	-1	1

g

0	1	-1
0	1	2
0	1	0

s

4	

Ejemplo de convolución 2-D

f

2	1	4	0
1	2	2	0
3	1	2	1
0	0	-1	1

g

0	1	-1
0	1	2
0	1	0

s

4	8

Ejemplo de convolución 2-D

f

2	1	4	0
1	2	2	0
3	1	2	1
0	0	-1	1

g

0	1	-1
0	1	2
0	1	0

s

4	8
5	

Ejemplo de convolución 2-D

f

2	1	4	0
1	2	2	0
3	1	2	1
0	0	-1	1

g

0	1	-1
0	1	2
0	1	0

s

4	8
5	5

Ejemplo de convolución 2-D

f

2	1	4	0
1	2	2	0
3	1	2	1
0	0	-1	1

g

0	1	-1
0	1	2
0	1	0

s

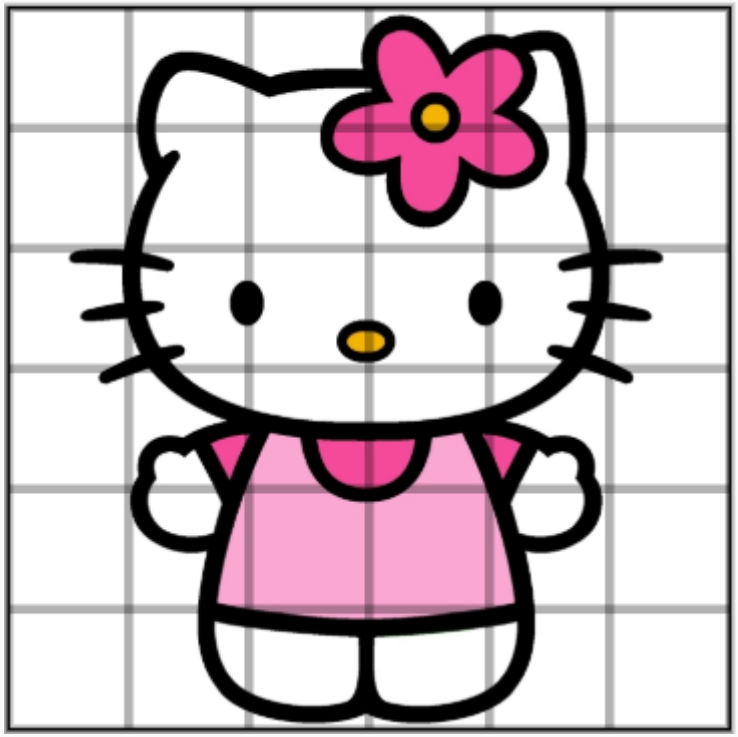
4	8
5	5

Campo receptivo

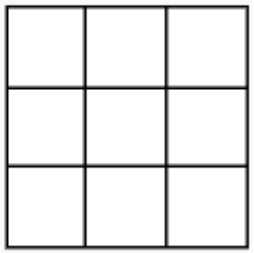
La salida de la operación tiene como objetivo la extracción de características de las distintas regiones de la imagen.

El campo receptivo de cada celda de la salida se activa cuando detecta una estructura de interés.

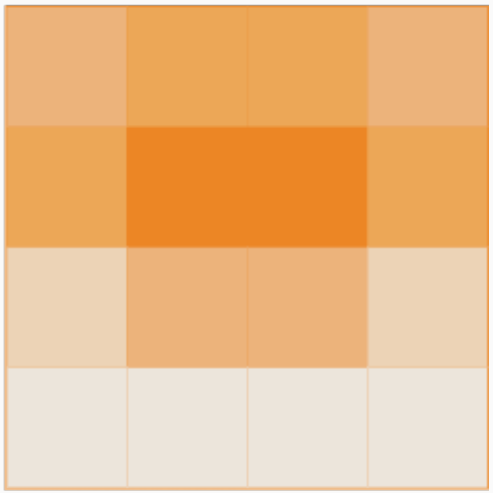
Input



Kernel



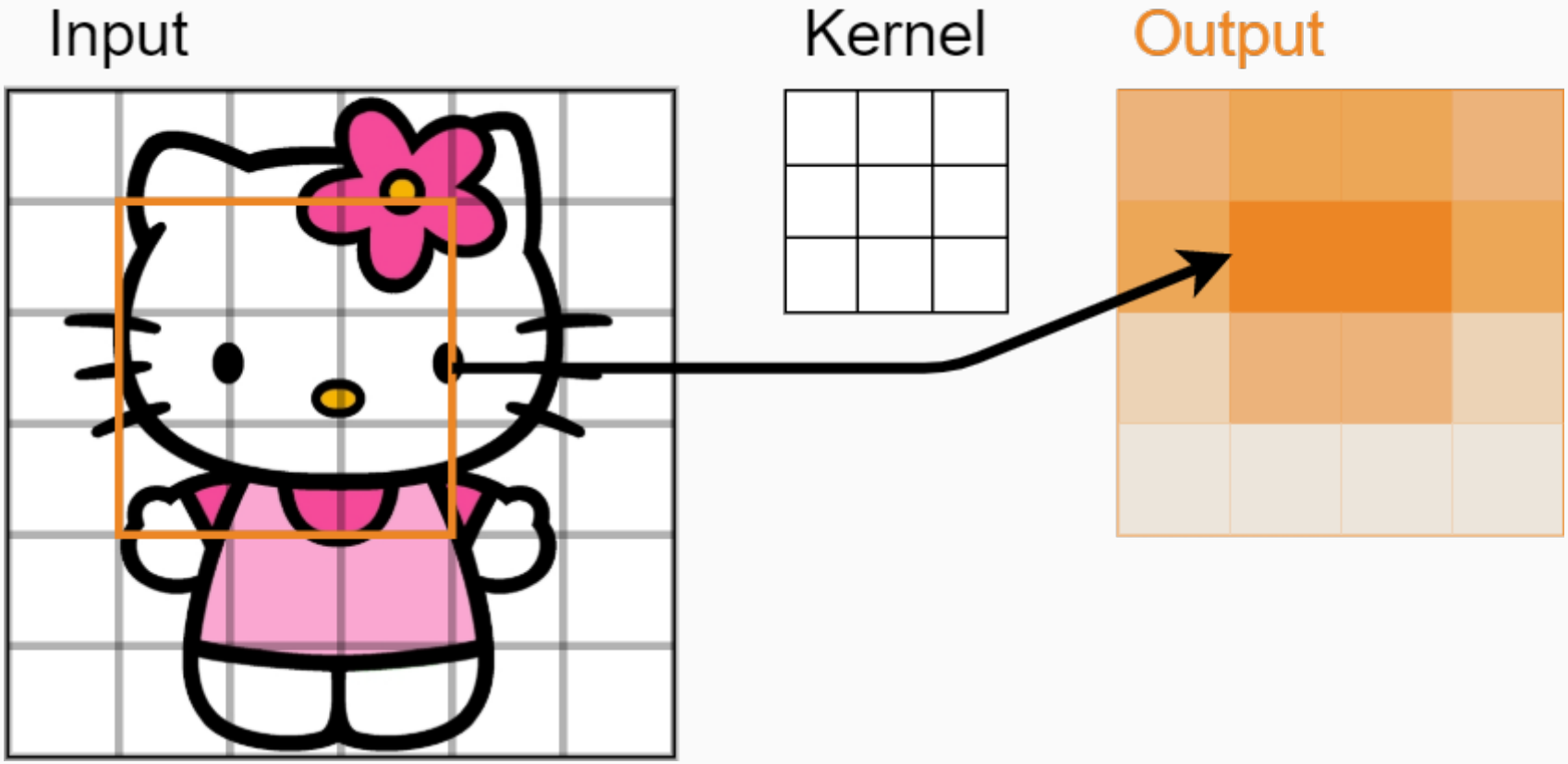
Output



Campo receptivo

La salida de la operación tiene como objetivo la extracción de características de las distintas regiones de la imagen.

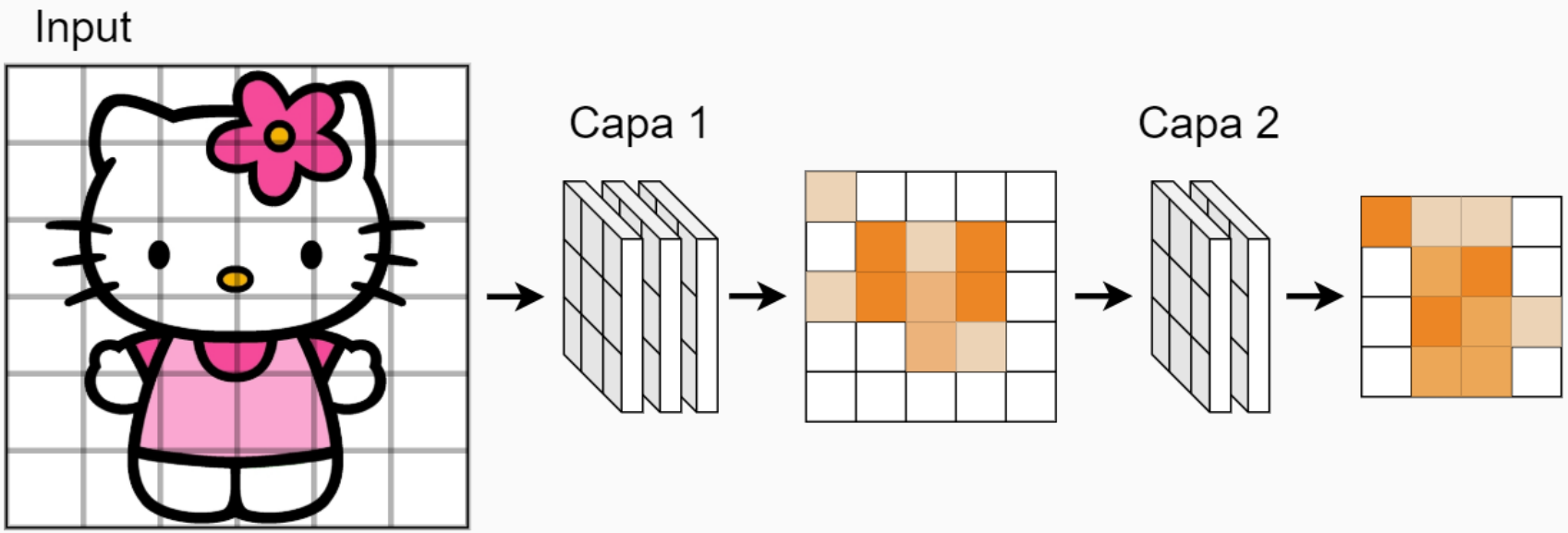
El campo receptivo de cada celda de la salida se activa cuando detecta una estructura de interés.



De neuronas a convoluciones

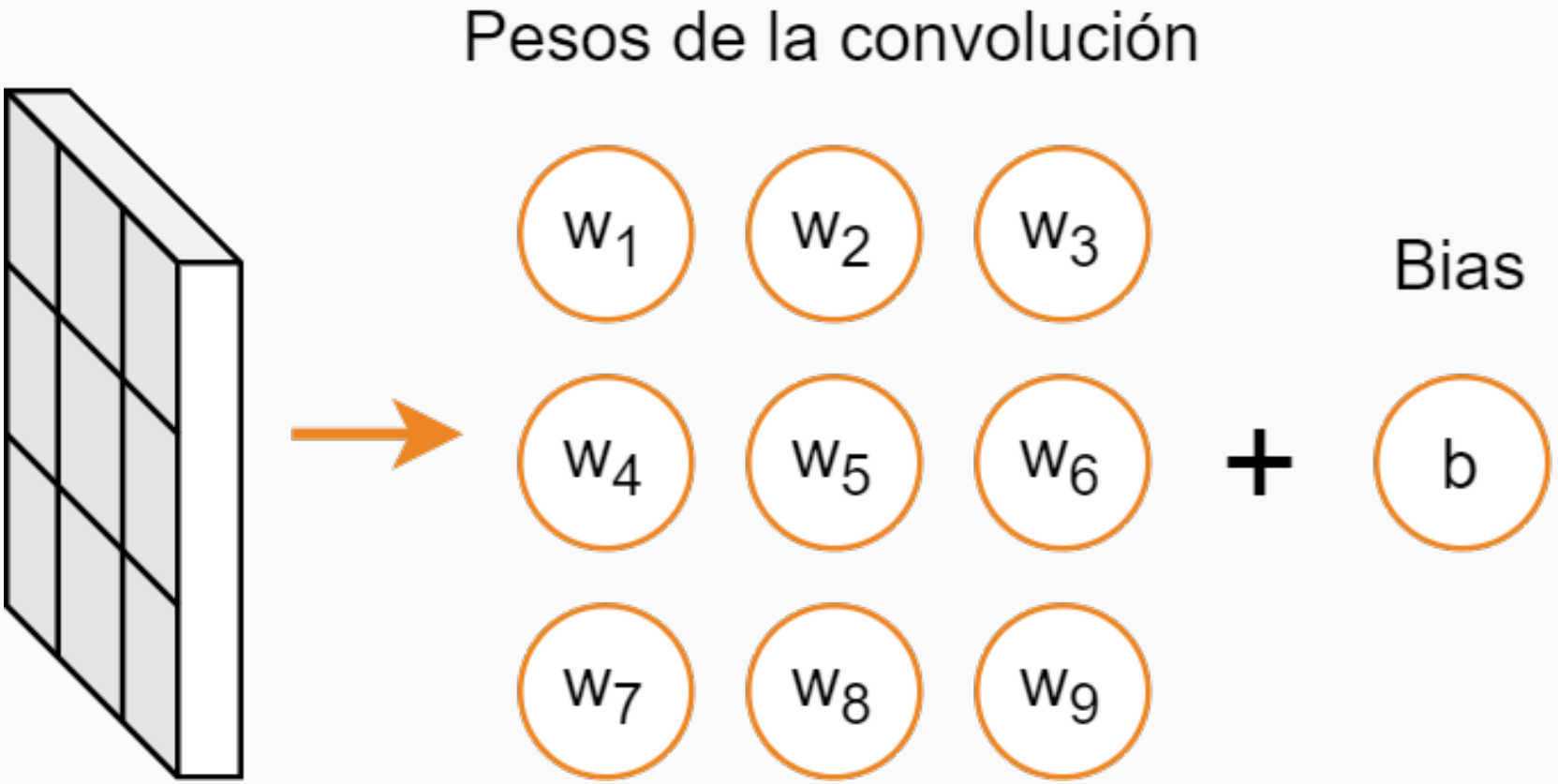
Una red de neuronas convolucional sustituye las capas densas por capas convolucionales.

Cada capa convolucional está compuesta por una serie de filtros de igual tamaño. Estos filtros se encargan de realizar el procesamiento de la información.



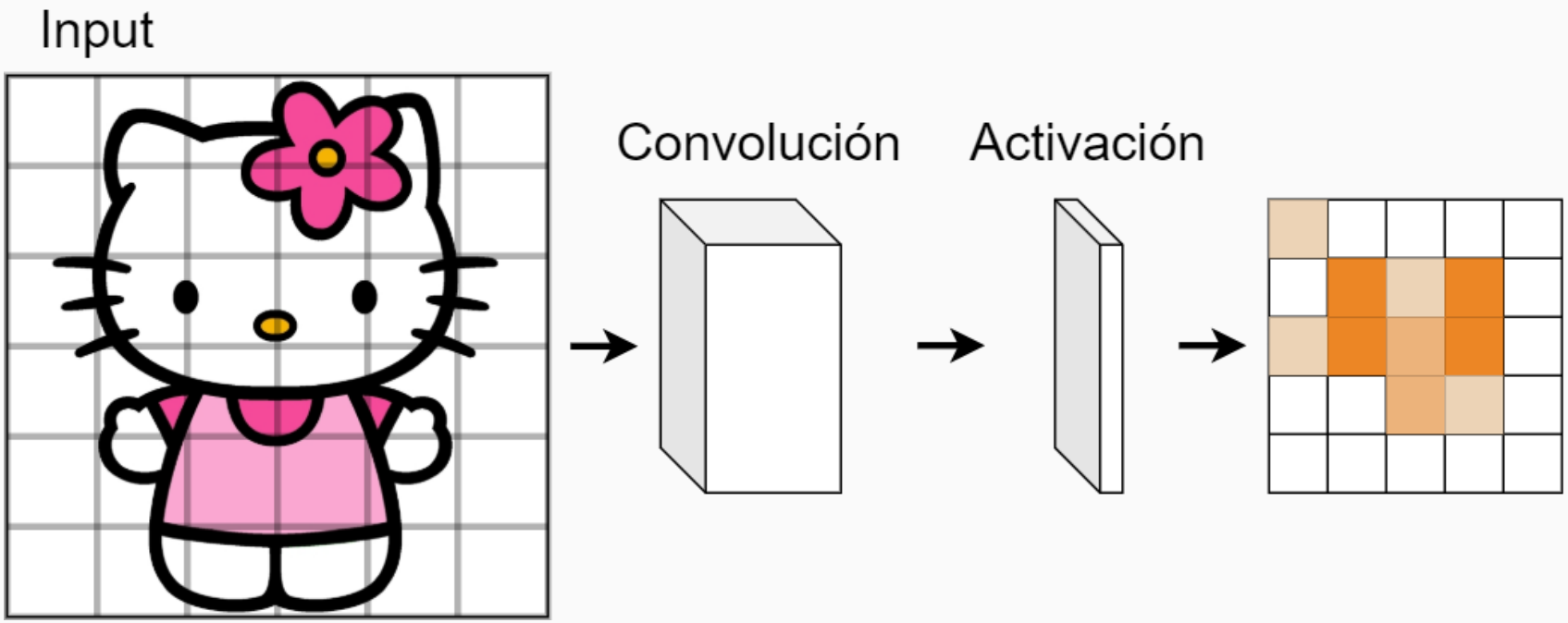
De neuronas a convoluciones

Cada filtro de la red está compuesto por una serie de neuronas.
Estas, igual que con las redes tradicionales tienen un peso asociado.
Este peso es el que regula cómo se realiza la convolución.



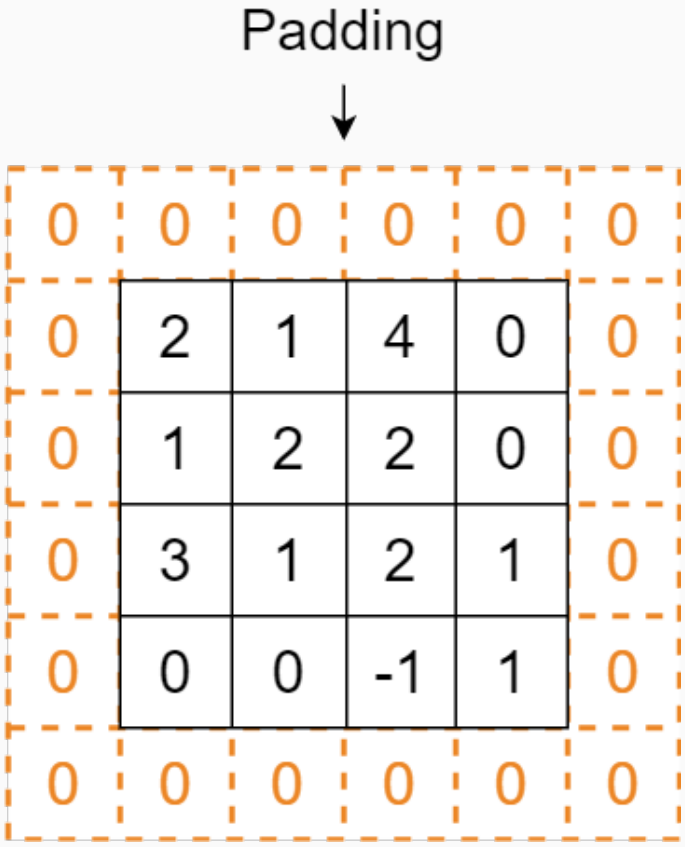
De neuronas a convoluciones

Tras haber realizado la convolución de unos datos de entrada, el resultado pasa por una activación a través de una función no lineal, tal y como sucede en las redes neuronales densas.



Padding en la convolución

Para controlar las dimensiones de salida de cada capa convolucional se aplica un *padding* a la imagen de entrada. Este consiste en un marco de "0" que evita la reducción dimensional.



Padding en la convolución

Existen dos configuraciones predominantes para la elección de padding en la librería keras:

- **Valid**: No se aplica ningún padding.
- **Same**: Se aplica un padding que haga que la dimensión de salida sea igual a la de entrada.

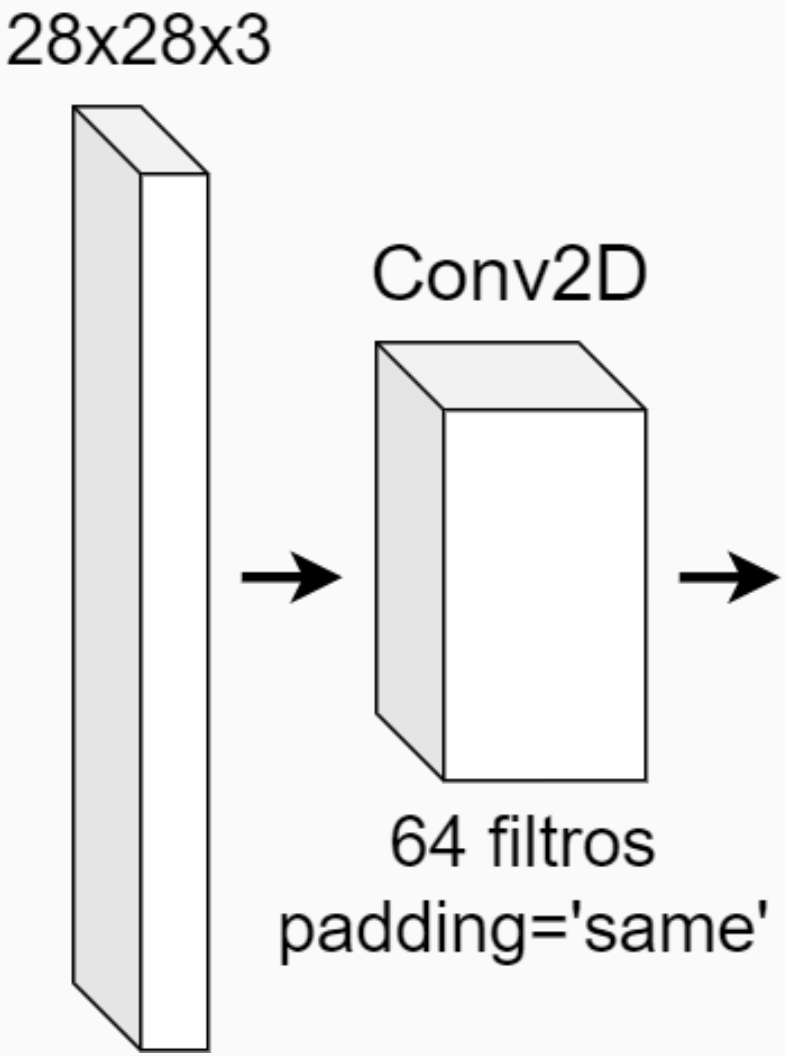
Ejemplo: Para una imagen de 16x16 píxeles y un filtro de 3x3, el padding "same" sería de 1 píxel.

Resultado de una capa convolucional

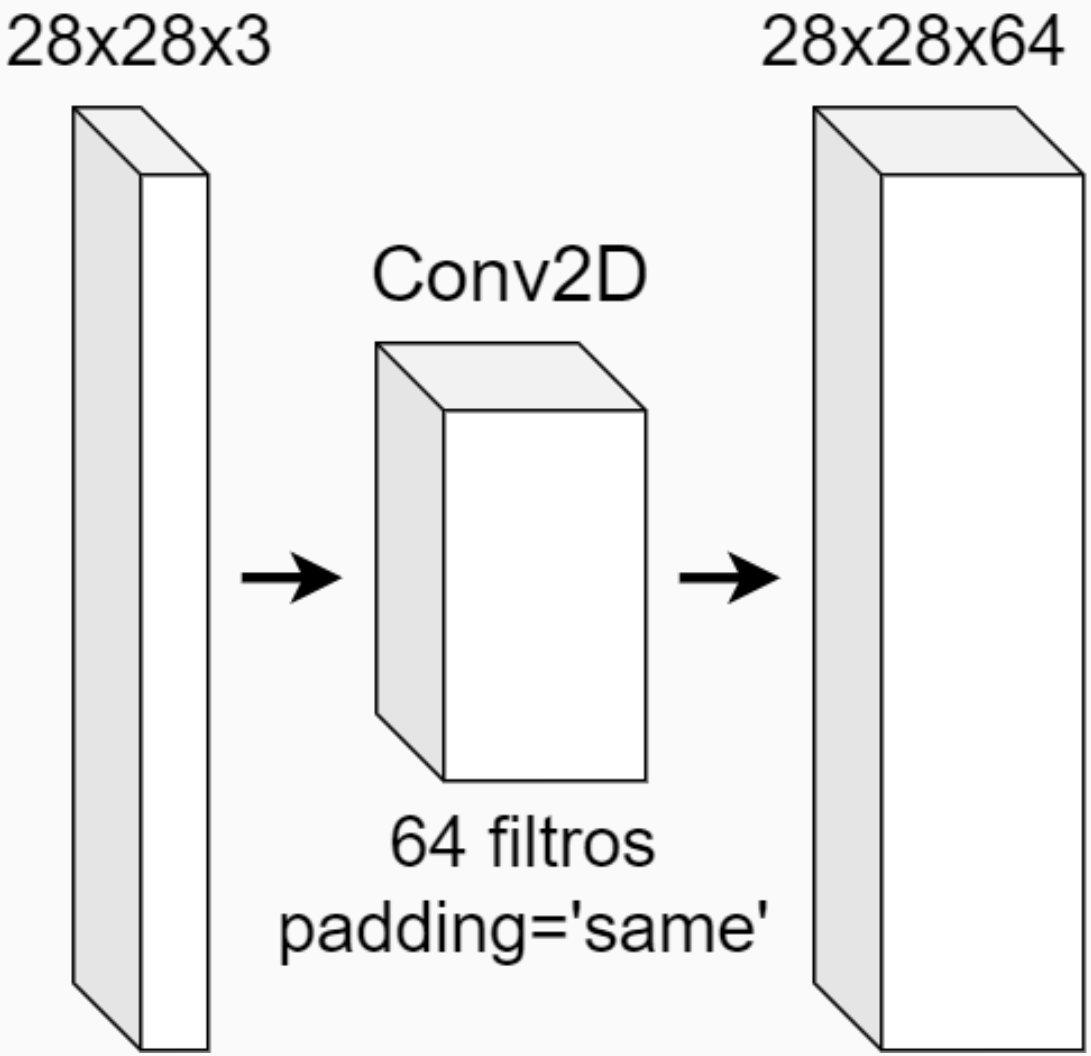
Una capa convolucional aplica varios filtros sobre la entrada y produce un volumen de salida cuyas dimensiones son:

- Alto y ancho: Determinados por las dimensiones de la entrada, el tamaño del kernel y el padding.
  - También influye el stride, se ve a continuación.
- Profundidad: Igual al número de filtros utilizados en la convolución.

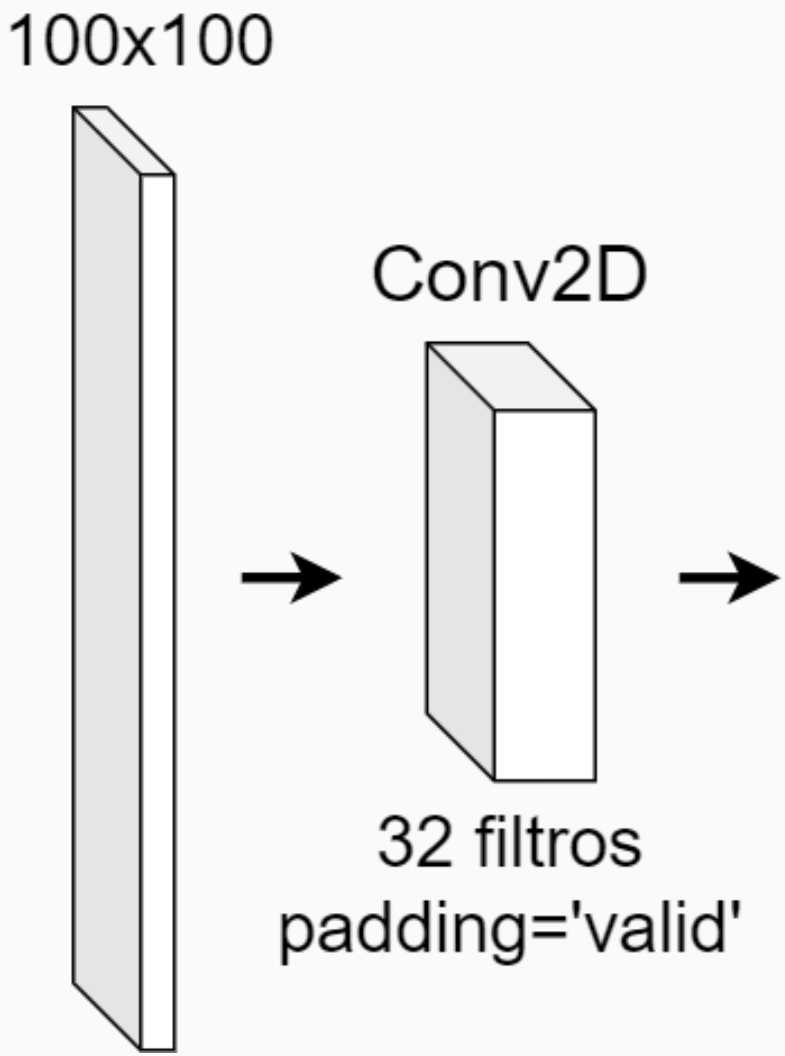
Resultado de una capa convolucional



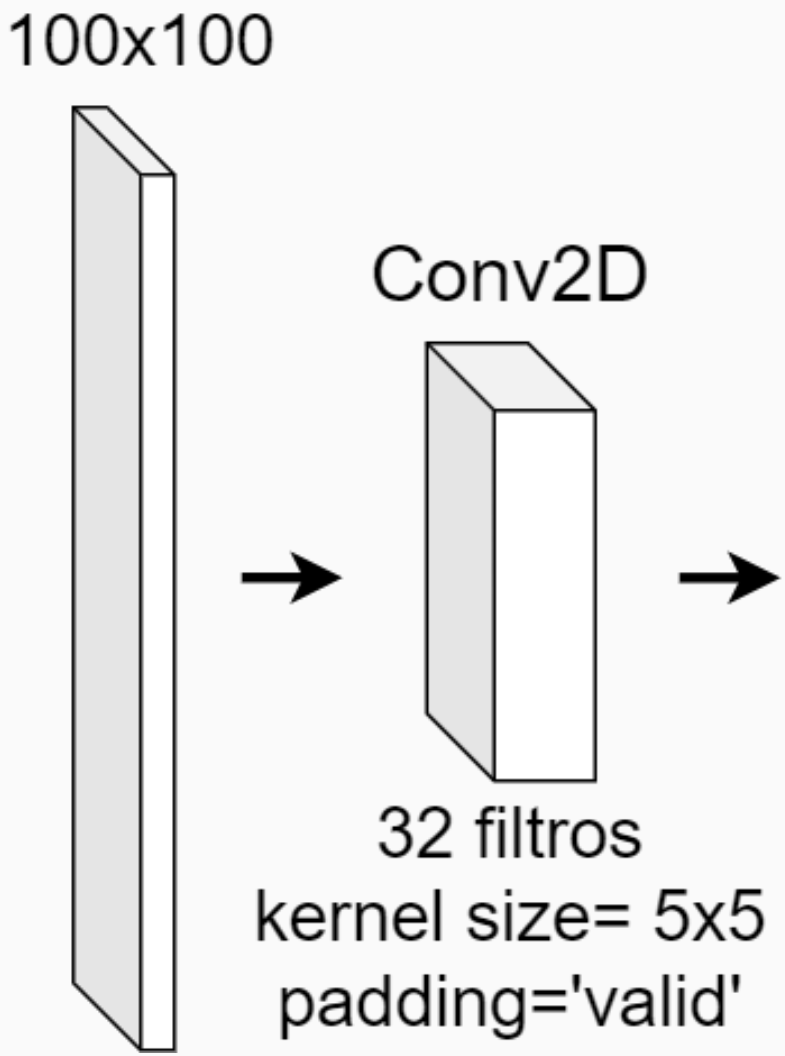
Resultado de una capa convolucional



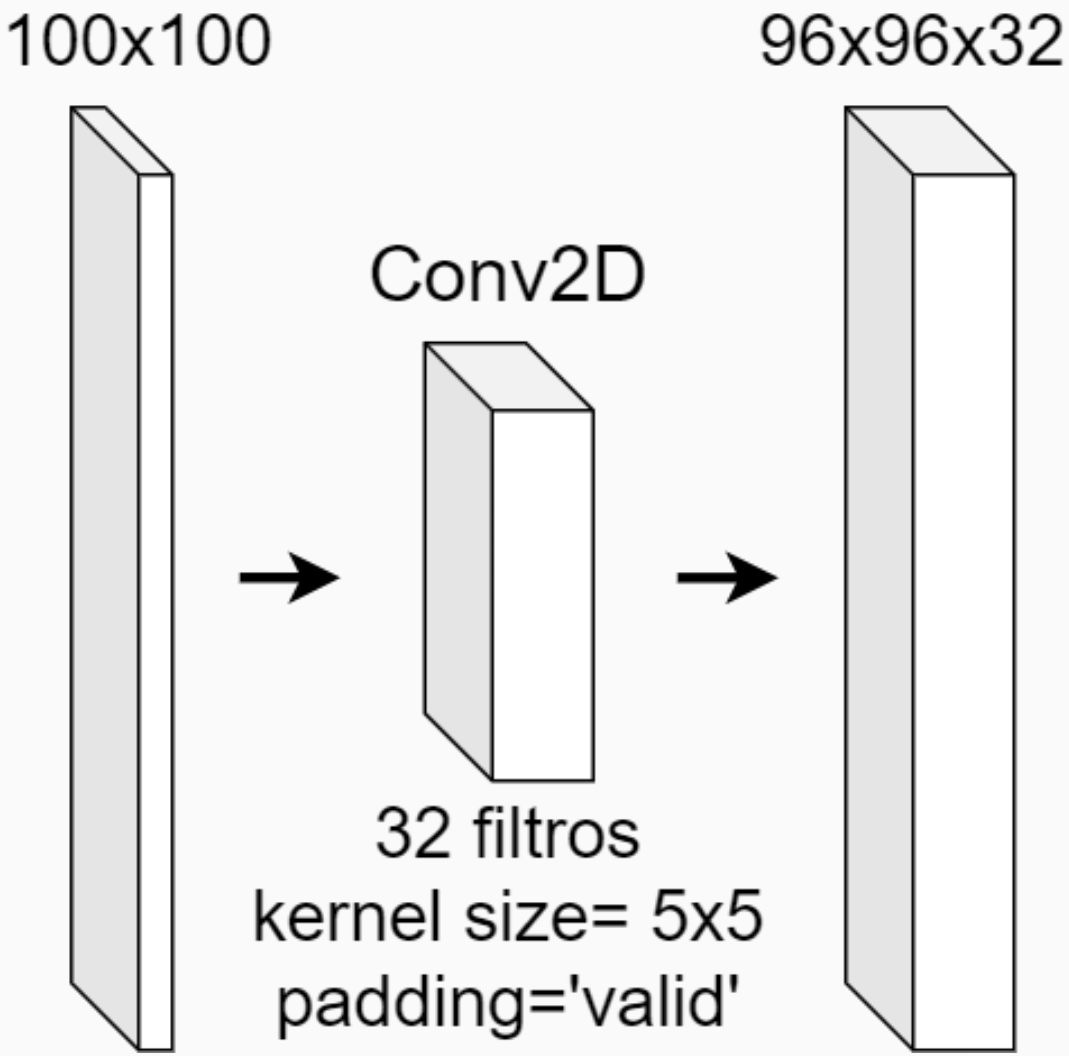
Resultado de una capa convolucional



Resultado de una capa convolucional



Resultado de una capa convolucional



Strides en la convolución

Los *strides* o pasos de una convolución corresponden con el número de casillas que se desplaza horizontal y verticalmente el filtro al realizar la convolución.

Stride = (2, 2)

f

2	1	4	0	2	3
1	2	2	0	1	2
3	1	2	1	0	1
0	0	-1	1	0	3
1	2	1	2	0	3
2	0	1	3	0	3

g

0	1	0
0	1	0
0	1	0

s

4	

Strides en la convolución

Los *strides* o pasos de una convolución corresponden con el número de casillas que se desplaza horizontal y verticalmente el filtro al realizar la convolución.

Stride = (2, 2)

f

2	1	4	0	2	3
1	2	2	0	1	2
3	1	2	1	0	1
0	0	-1	1	0	3
1	2	1	2	0	3
2	0	1	3	0	3

g

0	1	0
0	1	0
0	1	0

s

4	1

Strides en la convolución

Los *strides* o pasos de una convolución corresponden con el número de casillas que se desplaza horizontal y verticalmente el filtro al realizar la convolución.

Stride = (2, 2)

f

2	1	4	0	2	3
1	2	2	0	1	2
3	1	2	1	0	1
0	0	-1	1	0	3
1	2	1	2	0	3
2	0	1	3	0	3

g

0	1	0
0	1	0
0	1	0

s

4	1
3	4

Hiperparámetros de una convolución 2D

La capa Conv2D de la libería keras tiene una serie de hiperparámetros que permiten su configuración, dentro de los más importantes se encuentran:

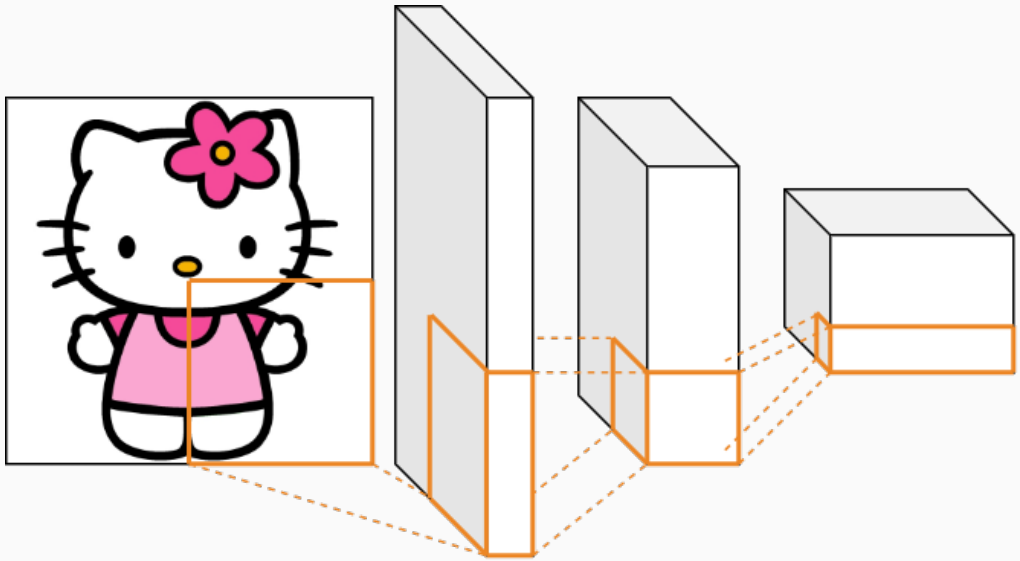
- filters
- kernel_size
- strides
- padding
- activation

Hiperparámetros de una convolución 2D

· filters

Corresponden al número de filtros que se le aplican a los datos de entrada.

Es un integer que corresponde con la profundidad del mapa de características de salida.

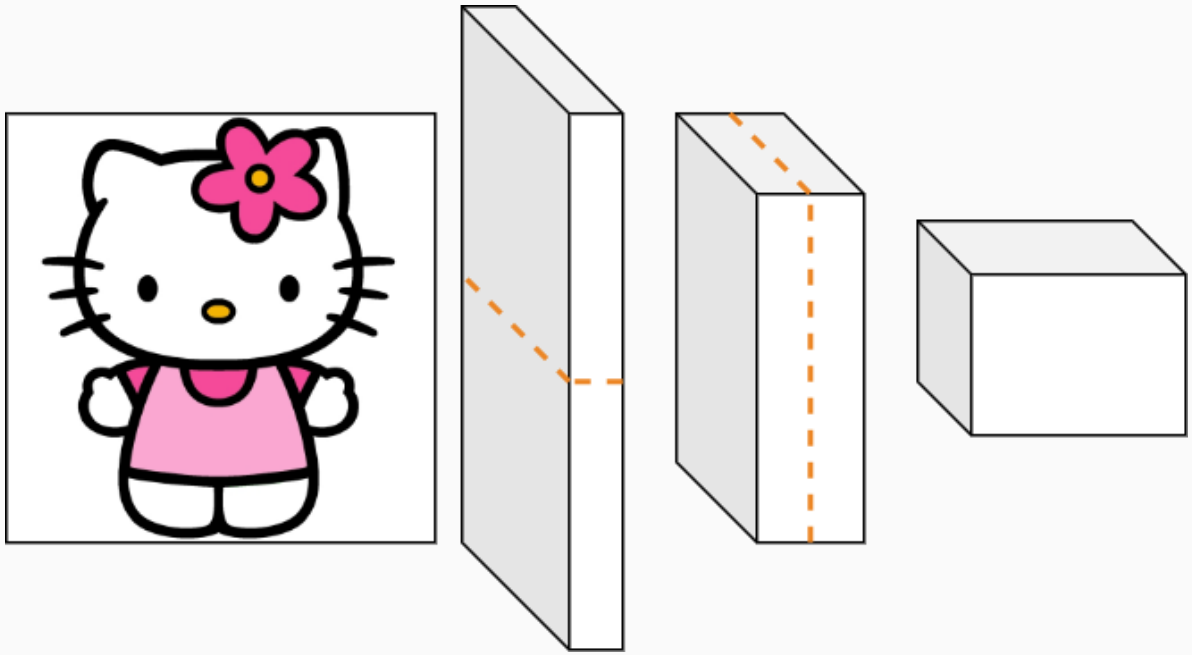


A medida que la imagen reduce su tamaño, los filtros aumentan. Se pretende extraer más características de alto nivel y cubrir zonas amplias de la imagen original.

Tamaño de imagen y filtros

Al mismo tiempo, a medida que el número de filtros de multiplica por 2 las dimensiones de la matriz de datos se reducen a la mitad.

El objetivo de este intercambio es mantener la misma cantidad de información, pero tratada por la red.



Hiperparámetros de una convolución 2D

- kernel_size

Determina el tamaño de los filtros que constituyen la capa.

Se define con un integer para filtros cuadrados, pero admite definir las dimensiones por separado en un vector (alto, ancho).

Habitualmente es de (3, 3) o (5, 5), en caso de imágenes muy grandes puede llegar a (7, 7).

Para matrices de datos más grandes se utilizan kernels más grandes, en casos combinando kernels de (5, 5) para las primeras capas y posteriormente (3, 3) para capas más profundas[1].

[1] Investigaciones posteriores [1] han demostrado que es más eficiente apilar dos capas convoluciones de (3, 3) que una única capa de (5, 5). Aunque esto sigue en debate [2, 3, 4] (los transformers revolucionan todo).

Hiperparámetros de una convolución 2D

- strides

Define el paso de la convolución a lo largo de los ejes.

Se define con un integer para un paso igual en ambos ejes, pero admite definir cada dimensión por separado en un vector (alto, ancho).

El paso de la convolución se mantiene a (1, 1) a no ser que se desee una reducción dimensional.

- padding

Determina el padding aplicado a los datos de entrada.

Se pueden definir las opciones "valid" y "same".

El padding de una convolución suele ser same para controlar las dimensiones de la matriz de datos, pero no es extraño encontrar casos con padding valid.

Hiperparámetros de una convolución 2D

- activation

Define la activación aplicada tras la convolución.

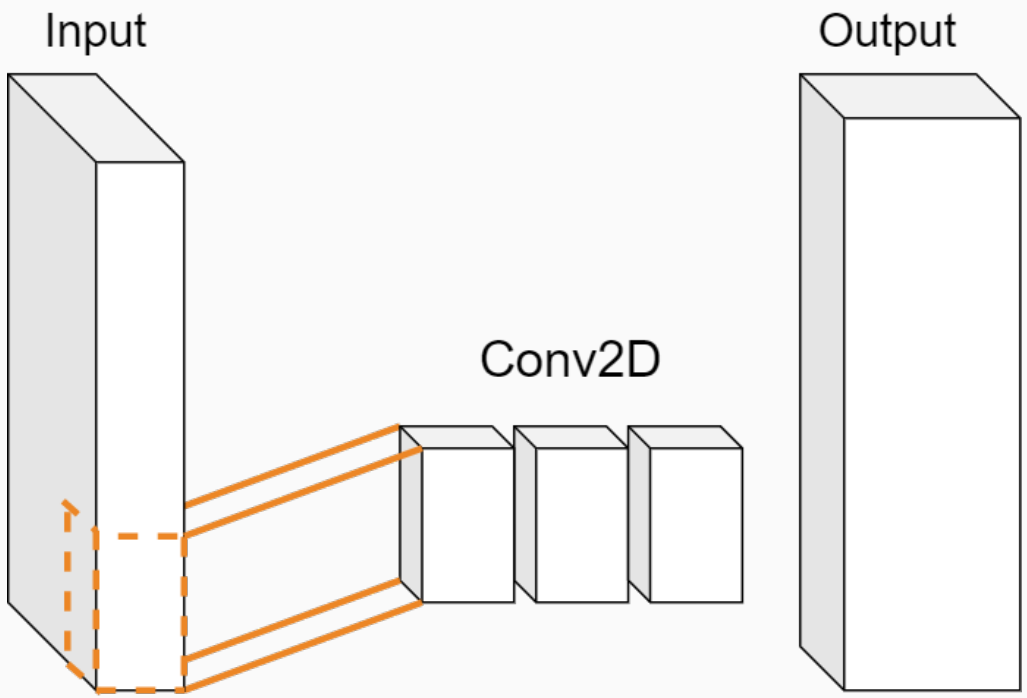
Dentro de las posibles activaciones se encuentran: relu, sigmoid, softmax, softplus, softsign, tanh, selu, elu, exponential.

Para las capas ocultas se suele utilizar la función ReLU o LeakyReLU, para la capa de salida la activación depende del problema concreto.

Parámetros de una convolución

El número de parámetros de cada capa convolucional viene dado por el tamaño del filtros, el número de filtros y la profundidad de la información de la capa anterior:

$$((kernel_{height} * kernel_{width} * depth_{input}) + 1) * filters \tag{1}$$



Notebook de ejemplo, dimensiones de convolución

El siguiente notebook contiene un breve código para explorar las dimensiones de salida de una capa convolucional.



- 1.2_01-DimensionesConv2D.ipynb

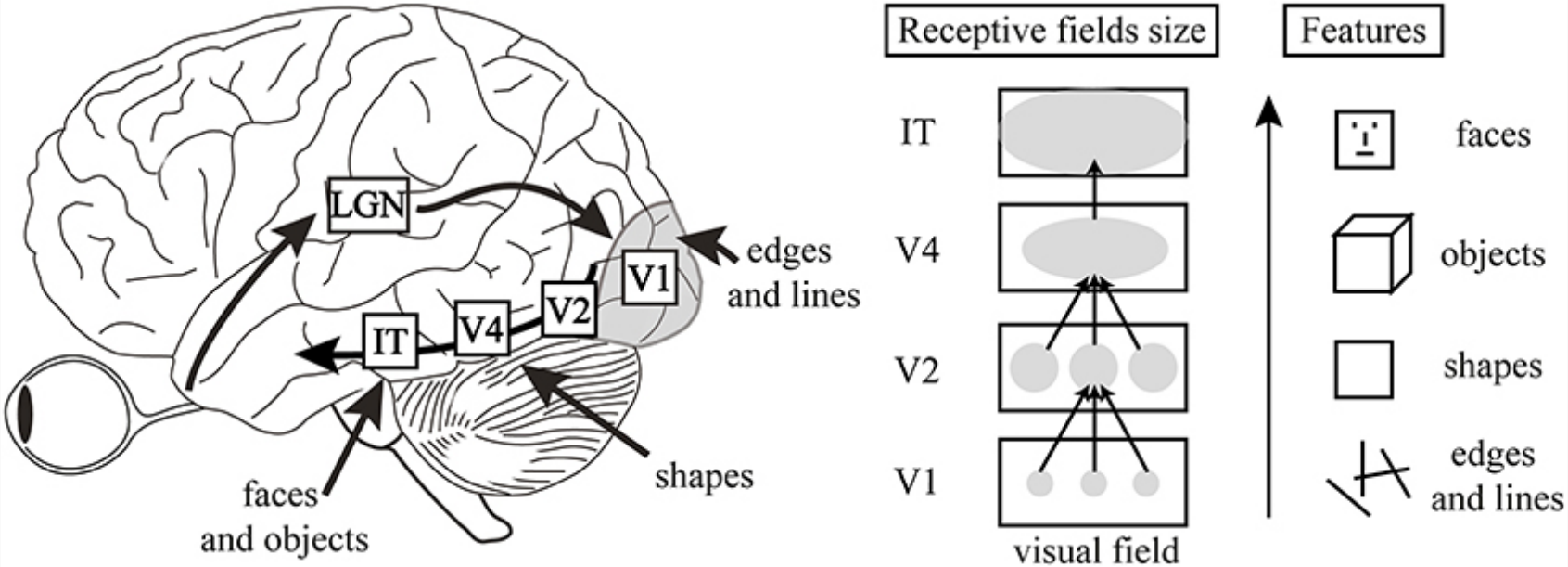
Aprendizaje de una red convolucional

A lo largo del tema se estudiarán distintas arquitecturas construidas con capas convolucionales, pero cabe destacar que la estructura por capas de estas redes consigue imitar el procesamiento del cortex cerebral del cerebro.

Las capas ocultas de las redes convolucionales contienen una jerarquía especializada en la tarea para la que se entrena.

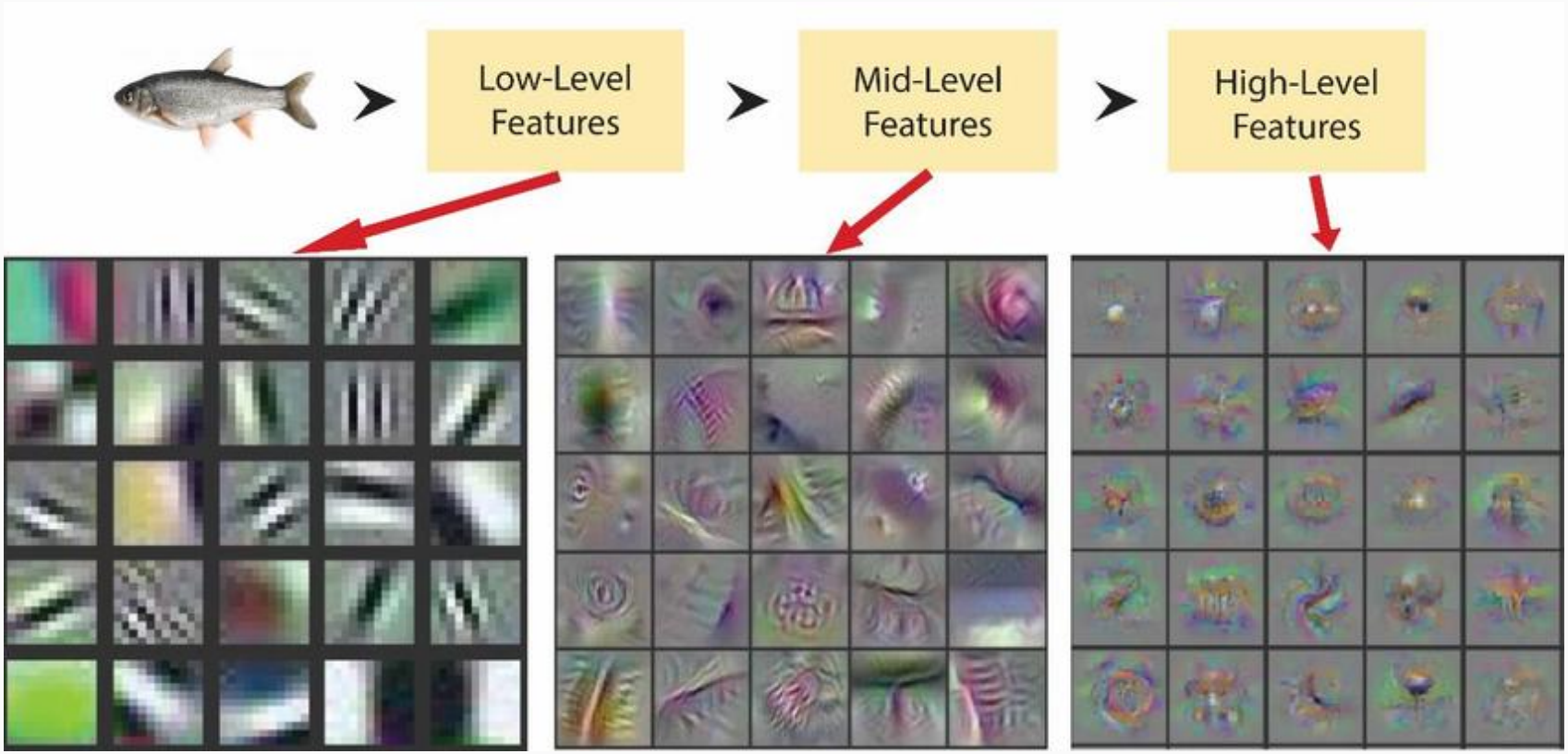
Esto se traduce en que, la primeras capas de la red se encargan de procesar información de bajo nivel, como líneas o curvas; mientras que las últimas capas se encargan de información de alto nivel, como una cara o la silueta de un animal.

Aprendizaje de una red convolucional



[5]

Aprendizaje de una red convolucional

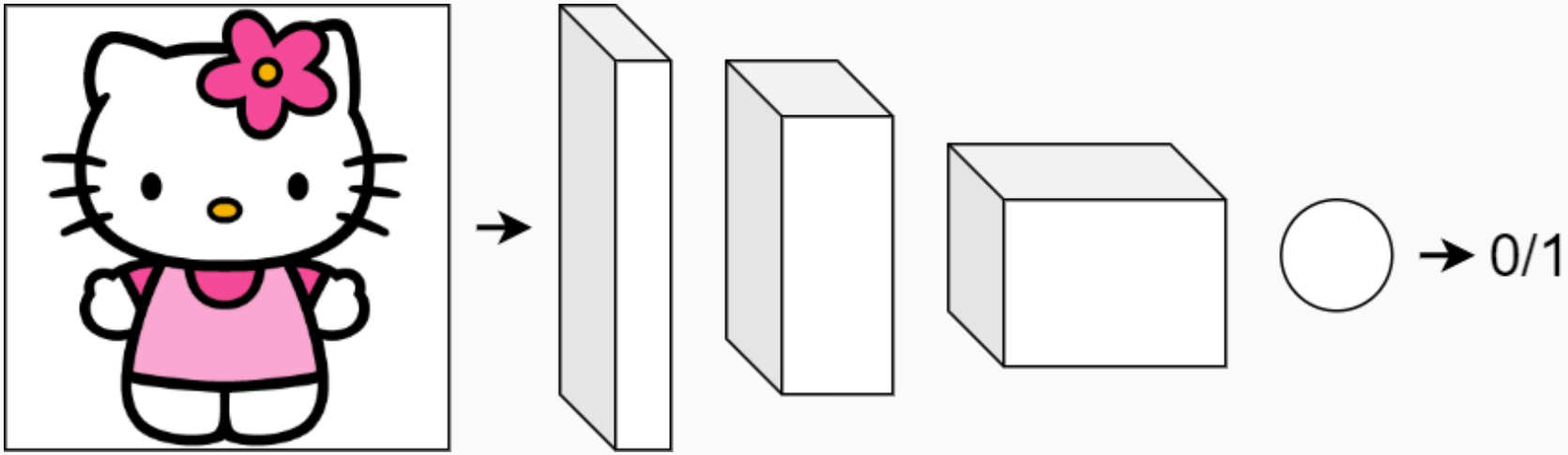


[6]

Construcción de una Convolutional Neural Network (CNN)

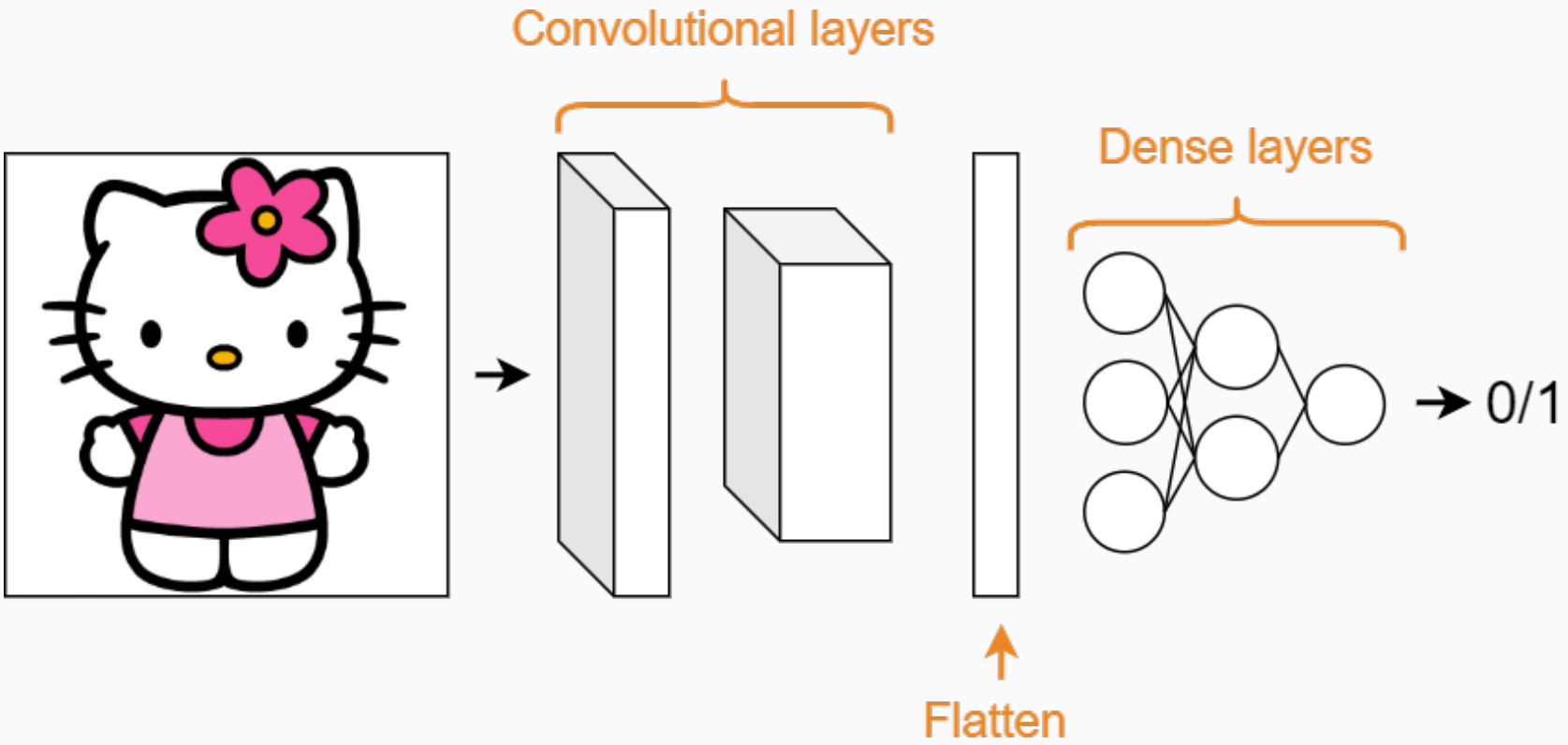
Construcción de una CNN

La estructura de embudo típica de las redes neuronales clasificadoras también se aplica a CNN. Para ello el objetivo es reducir la dimensión de la imagen hasta generar una salida.



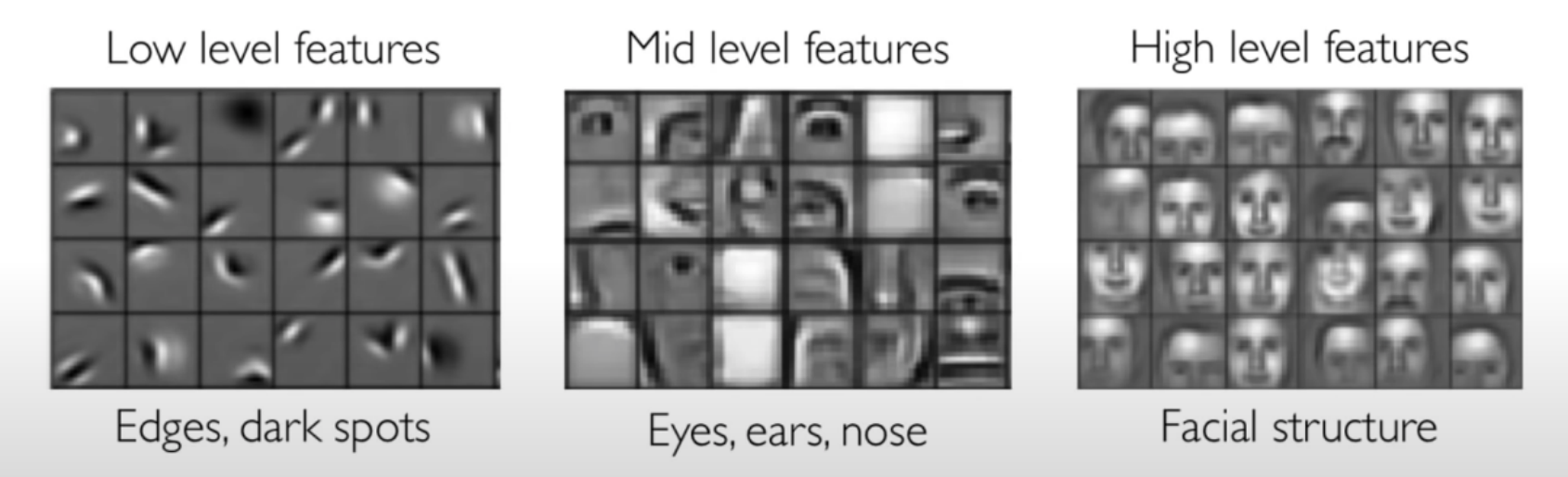
Construcción de una CNN

Las primeras capas convolucionales de la red se encargan de la extracción de características de la imagen. Posteriormente un perceptrón se encarga de clasificar las características extraídas para generar la salida deseada.



Construcción de una CNN

Es importante recordar que la jerarquía de capas de una red convolucional detecta características a alto nivel en las capas más profundas.



[7]

Reducción dimensional en redes convolucionales

Para formar el “embudo” de la red se utilizan distintos mecanismos para reducir las dimensiones de la información de la red. En concreto los dos mecanismos predominantes son:

- Capas de pooling
  - MaxPooling
  - AveragePooling
- Convoluciones de strides=(2,2)

Reducción dimensional con Pooling

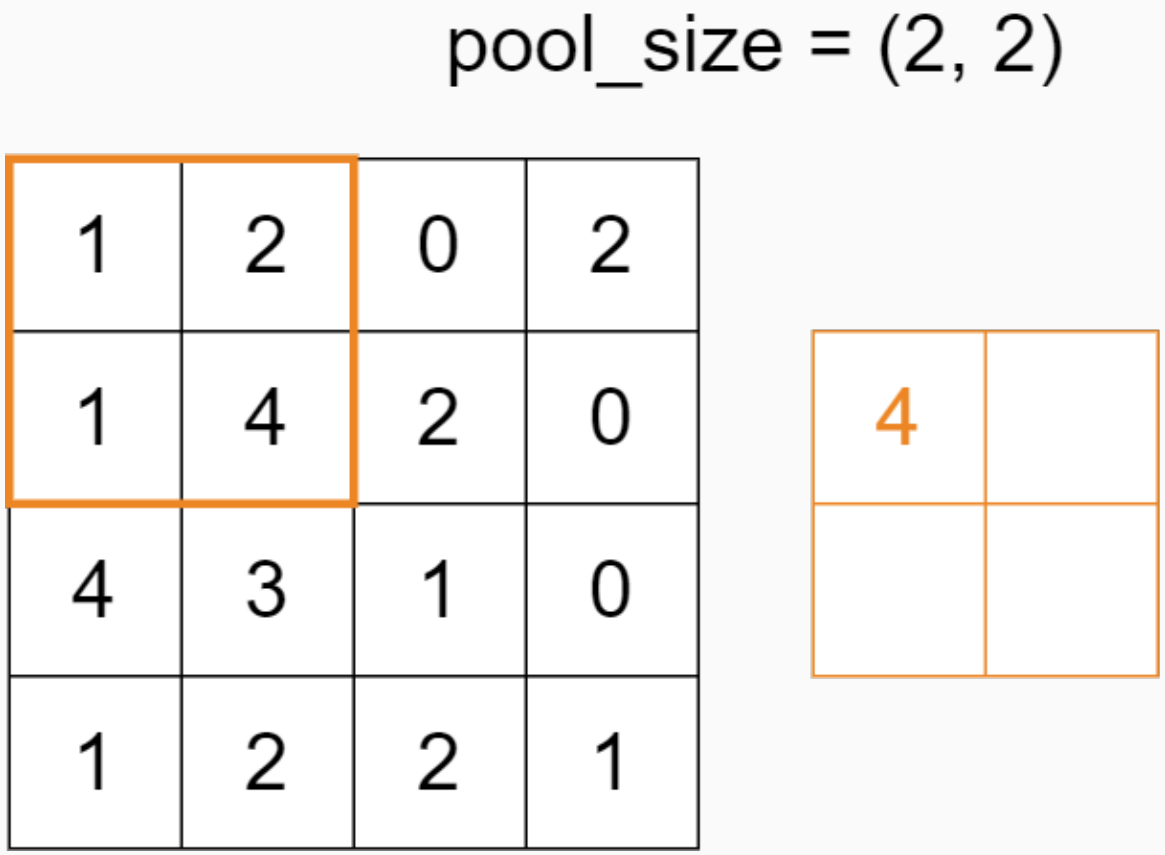
MaxPooling 2D La capa de MaxPooling2D reduce la dimensión de un vector cogiendo el máximo de la ventana definida.

pool_size = (2, 2)

1	2	0	2
1	4	2	0
4	3	1	0
1	2	2	1

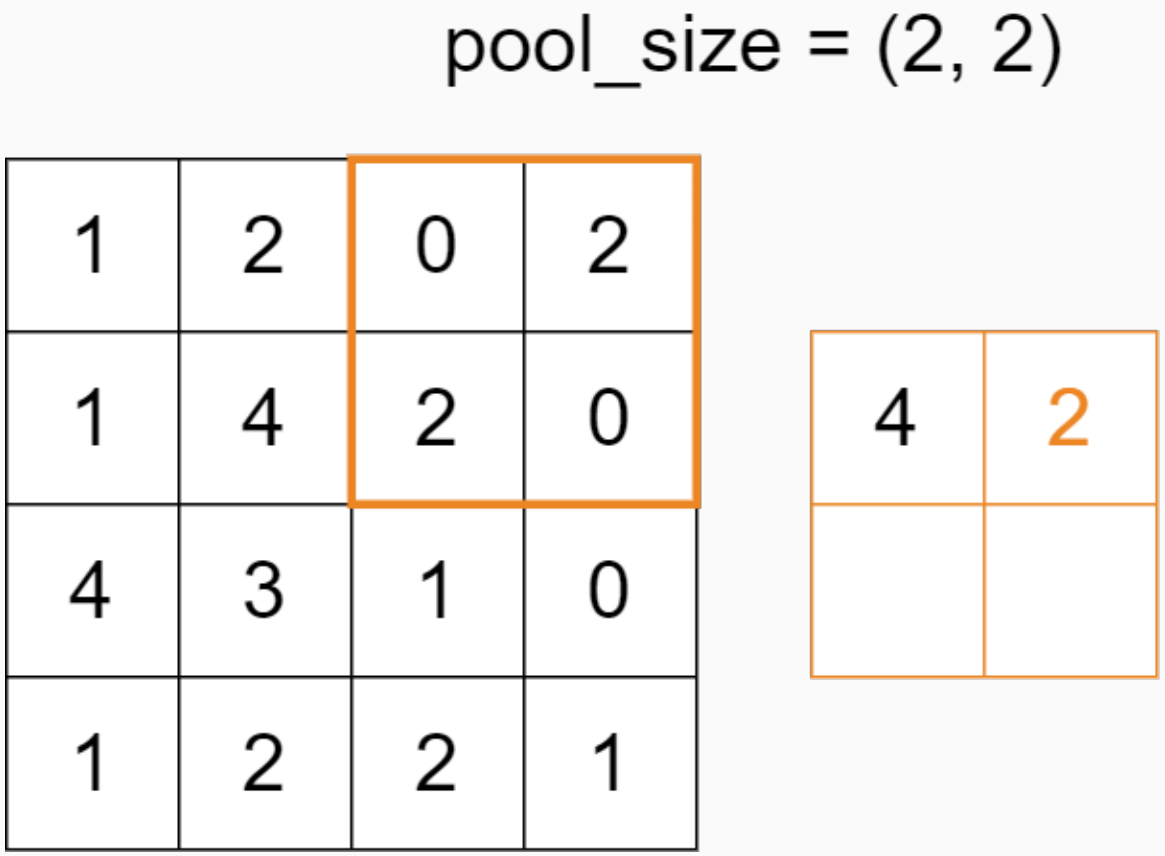
Reducción dimensional con Pooling

MaxPooling 2D La capa de MaxPooling2D reduce la dimensión de un vector cogiendo el máximo de la ventana definida.



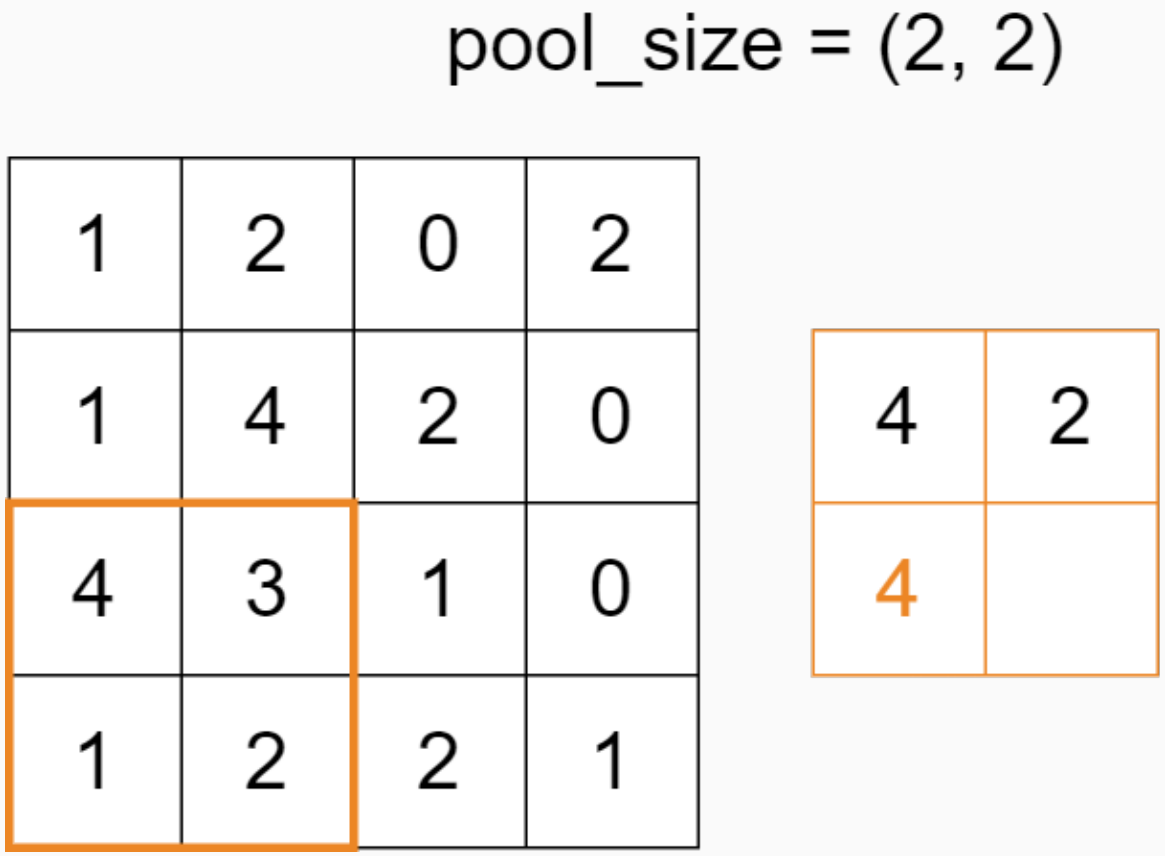
Reducción dimensional con Pooling

MaxPooling 2D La capa de MaxPooling2D reduce la dimensión de un vector cogiendo el máximo de la ventana definida.



Reducción dimensional con Pooling

MaxPooling 2D La capa de MaxPooling2D reduce la dimensión de un vector cogiendo el máximo de la ventana definida.



Reducción dimensional con Pooling

MaxPooling 2D La capa de MaxPooling2D reduce la dimensión de un vector cogiendo el máximo de la ventana definida.

pool_size = (2, 2)

1	2	0	2
1	4	2	0
4	3	1	0
1	2	2	1

4	2
4	2

Reducción dimensional con Pooling

MaxPooling 2D La capa de MaxPooling2D reduce la dimensión de un vector cogiendo el máximo de la ventana definida.

pool_size = (2, 2)

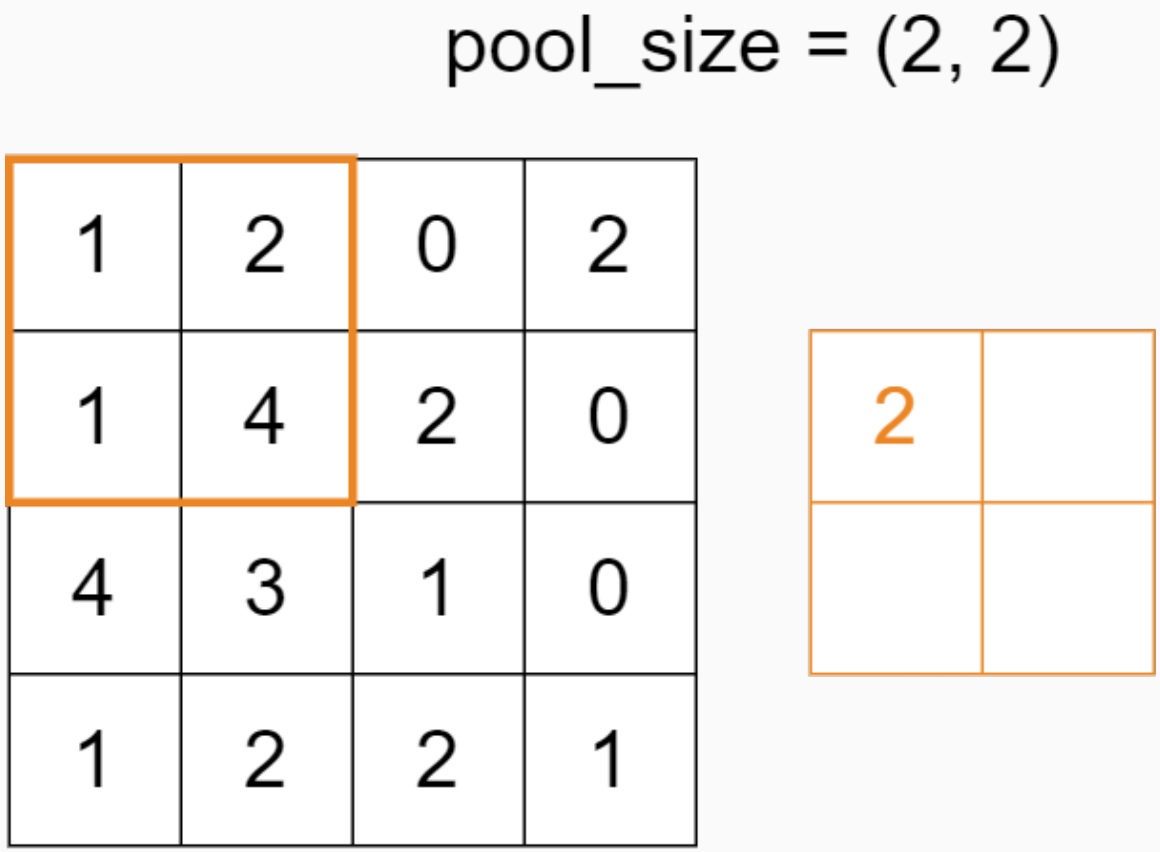
1	2	0	2
1	4	2	0
4	3	1	0
1	2	2	1

4	2
4	2

**cabe destacar que el máximo se encarga de preservar la característica más importante*

Reducción dimensional con Pooling

AveragePooling 2D La capa de AveragePooling2D reduce la dimensión de un vector cogiendo el promedio de la ventana definida.



Reducción dimensional con Pooling

AveragePooling 2D La capa de AveragePooling2D reduce la dimensión de un vector cogiendo el promedio de la ventana definida.

pool_size = (2, 2)

1	2	0	2
1	4	2	0
4	3	1	0
1	2	2	1

2	1

Reducción dimensional con Pooling

AveragePooling 2D La capa de AveragePooling2D reduce la dimensión de un vector cogiendo el promedio de la ventana definida[2].

pool_size = (2, 2)

1	2	0	2
1	4	2	0
4	3	1	0
1	2	2	1

2	1
2.5	1

[2] Pese a su similitar rendimiento, el MaxPooling suele generar mejores resultados que el AveragePooling [8]

Reducción dimensional con strides

Otra opción para la reducción dimensional es el uso de convoluciones con strides que no sean (1, 1).

- Cierta literatura apunta a esta aproximación como "más inteligente" ya que se le permite a la red que sea ella la que escoja cómo hacer la reducción.
- El principal inconveniente es que usando este método se aumenta el número de parámetros de la red.

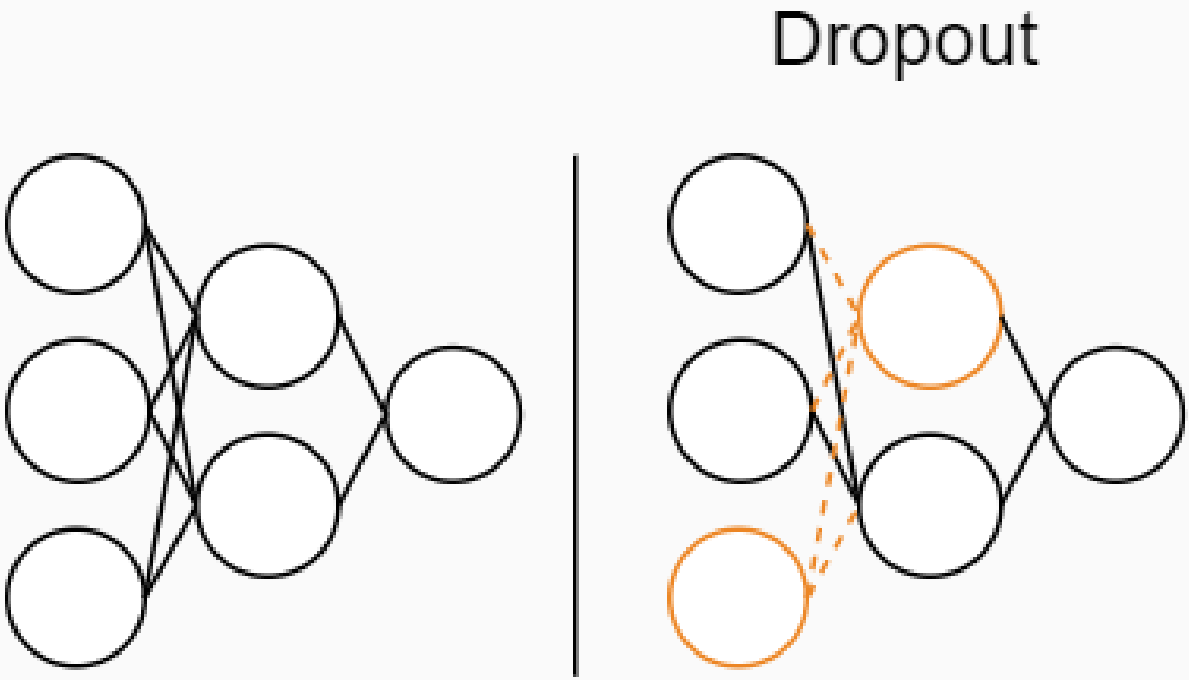
Existe un debate abierto sobre si es preferible que la red aprenda a realizar el *downsampling* [9], o si resulta más adecuado emplear técnicas como MaxPooling [10].

**Normalmente el stride elegido es de (2, 2) pero hay libertad para adaptarlo a distintos casos.*

Capa de dropout

La capa de dropout es una capa de regularización que desactiva aleatoriamente la activación de ciertas neuronas durante el entreno.

Al regularizar la red previene de problemas como el overfitting.



Notebook de ejemplo, clasificador con redes convolucionales

El siguiente notebook contiene un ejemplo de clasificador redes convolucionales.



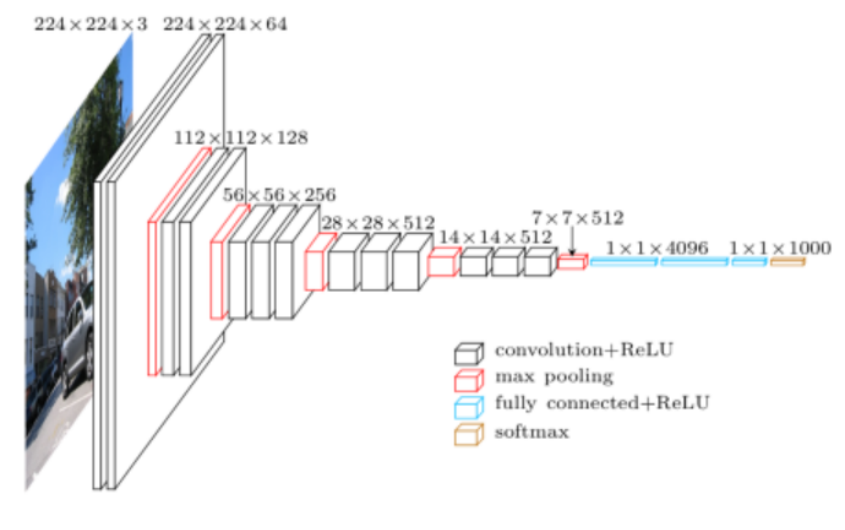
- 1.2_02-CNNImagenes.ipynb

Idea general

Usar un modelo preentrenado en un dataset grande (ej. ImageNet) para una nueva tarea.

- Enfoques comunes
  - Feature extraction: congelar capas y usar sus salidas como extractor de características.
  - Fine-tuning: reentrenar parcialmente algunas capas junto con las nuevas.
- Ventajas
  - Reduce el tiempo de entrenamiento.
  - Mejora los resultados con pocos datos.
  - Facilita el uso de arquitecturas complejas.

Ejemplo: VGG16



- VGG16
  - Red profunda propuesta por Simonyan y Zisserman (2014). *https://arxiv.org/abs/1409.1556*
  - Basada en bloques de convoluciones en embudo y luego densa.
- Uso en Transfer Learning
  - Importar el modelo preentrenado. Congelar sus pesos.
  - Sustituir densa por nueva densa.
  - Opcionalmente: descangelar algún filtro del final.

Ejemplo/Ejercicio



- 1.2_05-Cuadros.ipynb

Referencias i

[1] Karen Simonyan and Andrew Zisserman.
Very deep convolutional networks for large-scale image recognition.
arXiv preprint arXiv:1409.1556, 2014.

[2] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie.
A convnet for the 2020s.
In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11976–11986, 2022.

Referencias ii

[3] Xiaohan Ding, Xiangyu Zhang, Jungong Han, and Guiguang Ding.
Scaling up your kernels to 31x31: Revisiting large kernel design in cnns.
In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11963–11975, 2022.

[4] Shiwei Liu, Tianlong Chen, Xiaohan Chen, Xuxi Chen, Qiao Xiao, Boqian Wu, Tommi Kärkkäinen, Mykola Pechenizkiy, Decebal Mocanu, and Zhangyang Wang.
More convnets in the 2020s: Scaling up kernels beyond 51x51 using sparsity.
arXiv preprint arXiv:2207.03620, 2022.

Referencias iii

[5] Michael H. Herzog and Aaron M. Clarke (frontiers).
Cortex image.
https://www.frontiersin.org/articles/10.3389/fncom.2014.00135/full.

[Online; accessed August, 2022].

[6] Shoaib Ahmed Siddiqui, Ahmad Salman, Muhammad Imran Malik, Faisal Shafait, Ajmal Mian, Mark R Shortis, and Euan S Harvey.
Automatic fish species classification in underwater videos: exploiting pre-trained deep neural network models to compensate for limited labelled data.
ICES Journal of Marine Science, 75(1):374–389, 2018.

Referencias iv

[7] Kathrin Melcher (Medium).
Convolutional hierarchy image.
https://medium.com/low-code-for-advanced-data-science/introduction-to-convolutional-neural-networks-and-computer-vision-72b2d85dd1c0.
[Online; accessed August, 2022].

[8] Florentin Bieder, Robin Sandkühler, and Philippe C Cattin.
Comparison of methods generalizing max-and average-pooling.

arXiv preprint arXiv:2103.01746, 2021.

Referencias v

[9] itdxer (StackExchange).
Strides vs maxpooling debate.
https://stats.stackexchange.com/questions/387482/pooling-vs-stride-for-downsampling.
[Online; accessed September, 2023].

[10] Shuyang Sun, Jiangmiao Pang, Jianping Shi, Shuai Yi, and Wanli Ouyang.
Fishnet: A versatile backbone for image, region, and pixel level prediction.
Advances in neural information processing systems, 31, 2018.

Citación

Cita APA: Iglesias, G. and Talavera, E. (2026, noviembre 7). *Redes convolucionales* [Diapositivas de la asignatura Métodos Generativos]. GitHub. *https://guillermoih.github.io/M-todos-Generativos/Redes_Convolucionales.pdf*

Tema de las diapositivas "Metropolis" lincenciado bajo LPPL 1.3c

BibTeX

```
@slides{Redes_Convolucionales_MG_2026,
   author = {Iglesias, Guillermo and Talavera, Edgar},
   title = {Redes convolucionales [Diapositivas de la asignatura Métodos Generativos]},
   year = {2026},
   month = {11},
   howpublished={\url{https://guillermoih.github.io/M-todos-Generativos/Redes_Convolucionales.pdf}},
   note = {[Online; accessed 7 de noviembre de 2026]},
   organization={ETSISI. Universidad Politécnica de Madrid}
}
```